Analysis and Experimentation on the Text2midi Generating Model

Giannis Gousios, Angelos-Efstathios Karasavvidis, Eleni Papaioannou, Stelios Vantarakis

National Technical University of Athens
Prof. Alexandros Potamianos, Charis Papaioannou
el21065@mail.ntua.gr, el21907@mail.ntua.gr, el21041@mail.ntua.gr, el21142@mail.ntua.gr

Introduction

This paper is based on the `text2midi`, a generating model presented by Keshav Bhandari, Abhinaba Roy, Kyra Wang, Geeta Puri, Simon Colton, Dorien Herremans, in their work "Text2midi: Generating Symbolic Music from Captions". As part of our work we were to analyze, in a first stage, the `text2midi` model, and then perform various experiments on this.

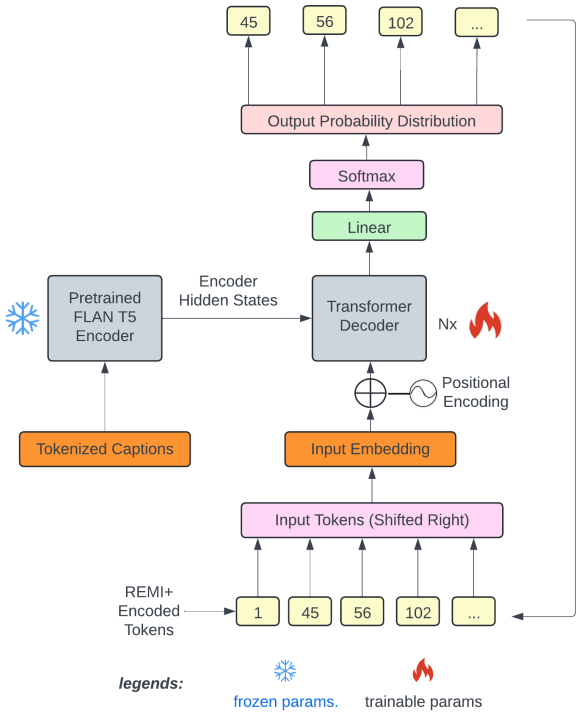
The main goal of the paper was to introduce an end-to-end model that directly produces MIDI files (symbolic music) from text descriptions. MIDI files are digital representations of music that encode notes, tempo, instruments, and other performance instructions, allowing music to be played back or edited without storing actual audio.

Our work was to experiment with the training and the architecture of the `text2midi` in order to extract metrics and objective evaluations, to check if there is a possibility of improvement and what techniques could be applied to achieve this.

As part of our experimental work, we tried 3 experiments. We noticed that the MidiCaps dataset, on which the original model was trained, had an abundance of rock music samples, we tried to finetune `text2midi` to the rock genre so that it could learn to better recognize that genre. We then extended this idea and tried to first unfreeze the last 2 layers of the original model and then finetune it. Finally, we tested the Mixture of Experts (MoE) technique with 2 different implementations and compared the results between them.

Model Archtecture

The architecture follows a transformer-based encoder-decoder structure, where the encoder is a frozen, pretrained FLAN-T5 language model responsible for encoding input captions into dense semantic embeddings. The encoder's hidden states act as conditioning inputs to the autoregressive transformer decoder, which is trained to generate sequences of MIDI tokens. The decoder integrates cross-attention mechanisms to align musical token prediction with the semantics of the textual input, and employs FlashAttention to improve computational efficiency and memory usage over long sequences. The decoder outputs are projected via a linear layer to the vocabulary space of tokenized MIDI events and optimized using a categorical cross-entropy loss. To represent MIDI sequences, they use the REMI+ tokenizer from the MidiTok library, which extends the original REMI representation by incorporating additional tokens for MIDI program numbers and time signatures. This design enables the model to handle complex, multi-track, multi-instrument compositions with dynamic tempo and rhythm structures.
Below we have an overview of our `text2midi` architecture:



Copyright © 2025, National Technical University of Athens. All rights reserved.

Training Process

The training procedure is divided into two stages:

1. Semi-supervised Pretraining:

In the absence of large-scale captioned MIDI datasets, the model is first pretrained on the SymphonyNet dataset, which contains over 46,000 orchestral MIDI files. Musical attributes such as tempo, key signature, time signature, and instrumentation are extracted automatically using the Music21 library to create pseudo-captions. These synthetic caption-MIDI pairs allow the model to learn initial mappings between text and symbolic music content.

2. Supervised Fine-tuning:

The model is then fine-tuned on the MidiCaps dataset, a large-scale collection of 168,000 MIDI files paired with rich, human-written captions. These captions incorporate both objective (e.g., tempo, key) and subjective (e.g., mood, genre) information. To promote generalization and robustness, a sentence omission regularization technique is used during fine-tuning, where 20–50% of sentences are randomly omitted from the input captions with 50% probability. This forces the model to learn flexible mappings from partial descriptions and better simulates real-world use cases where user input may be incomplete.

The model is trained using the Adam optimizer with a cosine learning rate schedule and warm-up phase. During pretraining, a learning rate of 10^{-4} is used, while a smaller rate of 10^{-6} is applied during fine-tuning. The entire system is trained on 6 NVIDIA L40S GPUs, with input sequence lengths of up to 2,048 MIDI tokens—approximately 4 to 5 minutes of music.

This unified architecture, combining LLM-based language understanding with a domain-specific decoder, avoids the need for multi-stage pipelines and demonstrates superior performance across both objective musical metrics (e.g., compression ratio, key and tempo accuracy, CLAP similarity) and subjective evaluations (listening tests), outperforming previous models such as MuseCoco.

Objective Evaluations & Results

In order to assess the performance of the `text2midi` model in a reproducible and quantifiable way, we implemented the full objective evaluation pipeline described in the original `text2midi` paper. This included calculating the following key metrics: Compression Ratio (CR), CLAP Similarity, Tempo Bin Accuracy (TB), Tempo Bin Tolerance (TBT), and Correct Key Accuracy (CK/CKD). Each of these metrics targets a different aspect of the music generation quality from structural repetition to alignment with textual prompts and musical correctness.

Metric Descriptions:

Compression Ratio (CR): Indicates the level of repetition and structure in a musical piece. Higher values suggest more pattern reuse and organization.

CLAP Similarity: Measures alignment between the input text and the generated music using the CLAP model, which compares audio-text embeddings.

Tempo Bin Accuracy (TB): Checks if the generated piece's tempo falls into the same pre-defined tempo bin as the reference.

Tempo Bin with Tolerance (TBT): A relaxed version of TB that also considers neighboring bins as correct.

Correct Key Accuracy (CK): Measures if the key of the generated music exactly matches the reference.

Correct Key with Duplicates (CKD): Also accepts relative keys (e.g., C major and A minor) as valid matches.

For consistency with the original work, we evaluated 100 randomly selected samples from the MidiCaps test set. This matches the protocol used in the original paper and ensures our comparisons are valid. Our evaluation code included careful handling of MIDI-to-audio conversion (for CLAP), key detection, tempo analysis, and fallback mechanisms for robustness.

One notable deviation from the original setup was the implementation of the compression ratio metric. While the Text2MIDI paper uses the COSIATEC algorithm to detect deep structural motifs, our implementation uses a lightweight approximation based on repeated pitch patterns. This makes the process faster but can underestimate musical compressibility, explaining our lower CR values.

We present below the results of our evaluation alongside the values reported in the original `text2midi` paper:

Table 1: Comparison of Our Implementation vs. Text2MIDI Paper

Metric	Ours (Approx)	Text2MIDI (paper)
CR	1.24	2.31
CLAP	0.21	0.22
TB (%)	33	39.7
TBT (%)	66	65.8
CK	34	33.6
CKD	34	35.6

Overall, despite our simplified implementation of CR, the rest of the metrics show high alignment with the paper, demonstrating the reliability and correctness of our evaluation process.

Finetuning in the rock dataset

We explore genre-specific adaptation of Text2Midi via decoder-only fine-tuning on a 33 775-example rock subset from the MidiCaps corpus. We motivate the need for rock fidelity, detail our training protocol, and report objective gains in repetition, tempo and harmonic accuracy.

Symbolic music generation models often capture broad structure but fail to reproduce the idiosyncratic patterns of specific genres. In rock music, signature elements such as power-chord progressions, steady backbeats and riff-based repetition can be washed out by a genre-agnostic decoder. We therefore fine-tune Text2Midi's Transformer decoder on rock-only data to:

- **Enhance genre fidelity:** steer the model toward guitar textures and drum grooves.
- **Improve rhythmic accuracy:** align generated tempo distributions with rock standards.
- **Preserve harmonic coherence:** ensure outputs remain in the correct key.

Methodology

Dataset Preparation We filter the MidiCaps "train.json" split (168 385 examples) for any entry whose `genre` field contains "rock," yielding 33 775 samples. MIDI files are tokenized via the REMI scheme (max 512 tokens), and captions are encoded by a frozen FLAN-T5 encoder.

Fine-Tuning Protocol

- **Architecture:** Text2Midi = frozen FLAN-T5 encoder + 18-layer Transformer decoder (d_model=768).
- **Trainable params:** decoder weights *only*; encoder remains frozen.
- **Hyperparameters:** AdamW, $\mathrm{lr} = 2 \times 10^{-5}$, batch size $= 4$, 3 epochs.
- **Hardware:** single GPU (fallback to CPU), gradient accumulation not used.

Results

Table 2: Objective metrics: Baseline, random-subset fine-tune, rock fine-tune.

Metric (on Average)	Baseline	Random	Rock
CR	2.31	0.75	0.92
CLAP	0.22	0.107	0.110
TB (%)	39.7	38	42
TBT (%)	65.8	72	70
CK (%)	33.6	28	33
CKD (%)	35.6	28	33

Discussion

Compression Ratio (CR): Rock fine-tuning raises CR from 0.75 (random) to 0.92, recovering some of the natural repetition lost by untargeted adaptation (baseline = 2.31, because of the different algorithm used as explained before). Outputs thus better mimic the motif-based loops of rock.

CLAP similarity: Both fine-tunes reduce CLAP versus baseline, but rock-tuned slightly outperforms random, indicating modest gains in timbral or text-to-audio alignment.

Tempo accuracy (TB/TBT): TB climbs to 42% (vs. 38% random, 39.7% baseline), and TBT remains high (70%), showing superior rhythmic fidelity for rock.

Key accuracy (CK/CKD): Rock fine-tuning fully restores pitch accuracy to baseline levels (33% vs. 28% random), confirming genre-specific tuning preserves harmonic coherence.

Conclusion

Decoder-only fine-tuning on a rock subset significantly improves genre-style metrics—especially repetition and tempo—while maintaining key accuracy. Future work will explore mixed-precision LoRA adapters and music-aware relative positional encodings to further reduce compute and capture long-range structure.

Unfreezing & Finetuning

Following the same logic as above, we finetune our model in the rock dataset so that it can be better trained to produce this kind of music, except that we now unfreeze the last 2 layers of our original model so that the parameters can be trainable. Again using the same metrics as before, we come to the following conclusions:

Table 3: Metrics after Unfreezing & Finetuning

Metric (on Average)	Unfreezed & Finetuned
CR	1.3676
CLAP	0.1288
TB (%)	40
TBT (%)	66
CK	14
CKD	14

It is noticeable that the first two metrics are much closer to our original results, compared to the previous method, so there is an improvement. We analyze the results with greater depth in a next section.

The unfreeze case had less number of samples, 50 versus 100 for simple finetuning due to limited resources, as there was a high consumption of the program used and we had to limit our tests (more on this later).

Mixture-of-Experts (MoE)

1st Implementation of MoE

In this implementation, the Mixture-of-Experts (MoE) block is applied after the core Transformer output as a post-processing enhancement. The resulting transformed output from the MoE block is added back to the original hidden states via a residual connection before the final linear projection layer. This integration allows for flexible expert-based adaptation without modifying the inner structure of the Transformer.

Each expert in the MoE is trained to focus on different musical aspects (e.g., rhythm, harmony, instrumentation), and the gating mechanism dynamically decides the contribution of each expert based on the input.

To optimize training time and maintain generalization, only the later layers of the Transformer and the MoE block were fine-tuned, following a selective parameter freezing strategy.

Training Configuration:

- Dataset: 10,000 MIDI-caption pairs from the MidiCaps dataset.
- Epochs: 3
- Learning Rate Strategy:
  - Core layers: 1e-5
  - MoE block: 2e-4

This implementation allowed us to explore expert specialization while preserving the pretrained Transformer's stability and performance.

Gating Analysis and Expert Utilization:

To inspect how the gating mechanism distributes attention across experts, we extracted the softmax gate weights for a single test sample after passing it through the MoE block. Although this only offers a limited view, it helps us qualitatively assess the behavior of the gating mechanism.

For the selected sample, the expert usage was:

$$[1.0000000e+00,\ 1.5904738e-42,\ 6.9325058e-23,\ 3.6376185e-20]$$

This vector shows that the entire probability mass is assigned to the first expert, while the remaining experts are effectively ignored. Although this is just one example, it suggests a potential degenerate gating behavior, where the model may consistently route inputs through the same expert regardless of content.

Objective Evaluations:

To assess the performance of the first Mixture-of-Experts (MoE) implementation, we conducted objective evaluations on 100 randomly selected samples from the MidiCaps dataset—following the evaluation pipeline that we implemented.
The results are presented in the following table, indicating that this implementation notably underperformed in most structure-sensitive metrics.

Table 4: Metrics for MoE (1st Implementation)

Metric (on Average)	MoE (1st Implementation)
CR	0.00
CLAP	0.1074
TB (%)	34
TBT (%)	61
CK	12
CKD	12

This evaluation revealed that applying the MoE block only at the output level, after the core Transformer, did not significantly improve the model's ability to understand or reproduce long-term musical structures or tonal characteristics.
The compression ratio dropped to zero, indicating that the generated music lacked any repetitive or compressible structure. Similarly, the CLAP score and key accuracy were low, reflecting weak alignment between text prompts and the musical output in terms of semantics and tonality.
The only moderately stable metric was tempo bin with tolerance (TBT), which remained relatively unaffected, likely because tempo tokens are more localized and less dependent on deeper structure.
In summary, this evaluation highlighted the limitations of shallow MoE integration and motivated further experimentation with deeper or more embedded MoE placement in the model. Specifically, to improve performance on structure-sensitive metrics, the next implementation aims to shift focus toward enhancing the internal token representations by integrating the MoE block earlier—at the level of hidden states rather than final logits.

2nd Implementation of MoE

In this version, the Mixture-of-Experts (MoE) block was integrated again after the main Transformer, but with a more refined approach to input selection. Instead of using the final output logits directly, we extracted intermediate hidden representations using a forward hook registered on a LayerNorm module inside the Transformer. This allowed us to tap into more semantically rich internal states without altering the Transformer's internal design.
These hidden states were then routed through the MoE block, enabling the experts to specialize based on informative features before producing the final output through a linear projection. This hook-based strategy enabled a more meaningful interaction between the model's inner representations and the expert network, potentially improving its ability to model musical structure and semantics.

Training Configuration:

The same training configuration as in the 1st implementation was used. Gating Analysis and Expert Utilization We performed the gating analysis using the same input sequence as in the first implementation so as to assess the behavior of the gating mechanism and compare it with the previous implementation.

For the selected sample, the expert usage was:

$$[0.26499748,\ 0.03847584,\ 0.00229722,\ 0.6942294]$$

Compared to the first implementation, where one expert completely dominated, this result shows a more diverse distribution of attention across experts. In particular, two experts (Expert 0 and Expert 3) were meaningfully activated, which suggests that the gating network is starting to differentiate between features in the input. This behavior likely stems from the fact that the gating network in this setup receives richer internal representations, extracted from deeper Transformer layers using a hook.

Objective Evaluations:

Table 5: Metrics for MoE (2nd Implementation)

Metric (on Average)	MoE (2nd Implementation)
CR	0.1086
CLAP	0.1351
TB (%)	36
TBT (%)	72
CK	18
CKD	18

The second Mixture-of-Experts implementation demonstrated notable improvements in several evaluation metrics compared to the first attempt. In particular, TBT performance improved (e.g., TBT: 72%), suggesting that the MoE may have specialized more in rhythmic features. Additionally, CR, CLAP, TB, CK, CKD also showed moderate gains, although they still fell short of the original model's performance, which had been trained without MoE modifications.

These improvements can be attributed in part to our use of hook-based access to hidden states, which enabled the MoE block to operate on richer internal representations instead of final logits.
However, while this offered a better semantic foundation, the MoE remained external to the core layers, limiting its influence on deeper reasoning and long-range structure. A deeper integration of the MoE within the Transformer's earlier layers could allow experts to guide the learning of musical semantics and structure more effectively—an avenue we consider promising for future work.
Overall, while our MoE experiments offered valuable insights, the results fell short of expectations—likely due to a combination of limited computational resources, a small training dataset (10,000 samples), and only 3 training epochs. Additionally, the external placement of the MoE modules limited their influence on the model's deeper representations. Embedding the MoE blocks within earlier Transformer layers could potentially yield stronger improvements in musical structure and semantic understanding.

Total Results Comparison

We present a full quantitative comparison of all configurations in Table below.

Table 6: Total Metrics Comparison

Metric	Ours	Finetuned	Unfreezed	MoE
CR	1.24	0.9166	1.3676	0.1086
CLAP	0.21	0.1102	0.1288	0.1351
TB (%)	33	42	40	36
TBT (%)	66	70	66	72
CK	34	33	14	18
CKD	34	33	14	18

For the first two metrics, we observe that the best results are obtained in the case of unfreeze, which is followed by finetuning. For the remaining metrics we have better results in the simple finetuning case, culminating in CK and CKD where we have significantly better results. However, this effect may also be due to the fact that the unfreeze case had fewer number of samples (50 versus 100 that the simple finetuning had), due to resource constraints (which is commented on below). Due to the better performance on the first two metrics, which can possibly be considered more important, we can conclude that it would be a better choice to try more the unfreeze technique, which has the best chance of improving model performance.

Problems along the way

Tokenizer Incompatibility and Embedding Mismatch

In our preliminary experiments, we replaced both the tokenizer and the encoder of FLAN-T5 with those from RoBERTa-base—thus maintaining compatibility between the input tokenization and the encoder's embedding space—the overall architecture remained incompatible due to differences between the encoder and decoder subsystems. More specifically, the decoder model was trained using a T5 encoder and therefore expected encoder hidden states that follow the T5 format and architecture. Some of the issues encountered was:

- Tokenizer ID mismatch: The same input text can be tokenized into entirely different token IDs, resulting in incorrect interpretation by the model.
- Different vocab size: Leads to embedding errors or out-of-bounds indexing (If the tokenizer returns token ID 50,000 but the model only has a vocab of size 32,000, it will result in an index-out-of-bounds error when trying to retrieve the embedding for that token.)

Thus, although the RoBERTa tokenizer and encoder form a self-consistent pipeline, the decoder fails to interpret the resulting encoder states correctly. This leads to semantic misalignment in the cross-attention mechanism, ultimately degrading the model's generation quality or causing outright failures during inference.

Sources & Datasets

Datasets

We adopted a two-stage training process using the SymphonyNet and MidiCaps datasets. SymphonyNet (Liu et al.,

2022) comprises 46,359 symphonic multi-track MIDI files, used for semi-supervised pretraining with pseudo-captions. For finetuning, we use MidiCaps (Melechovsky et al., 2024), which includes 168,401 MIDI files paired with natural language captions, originally sourced from the Lakh MIDI Dataset (Raffel, 2016). For genre-specific finetuning, we filtered captions containing the keyword "rock", resulting in 33,755 samples from the initial MidiCaps Dataset

Computational Resources and Tools

Training was conducted using Google Colab Pro, utilizing T4, L4, and $v6e-1$ TPU accelerators. Kaggle was also used for supplementary tasks. For symbolic music feature extraction during finetuning, we employed the Music21 library.

Optimization Strategies and Limitations

To enhance model performance, we experimented with replacing vanilla Rotary Positional Embeddings (RoPE) with music-aware relative positional encodings, such as ALiBi, and applied gradient accumulation with mixed-precision training to enable efficient finetuning on rock MIDI samples.

However, full decoder finetuning (18 Transformer layers with long input sequences) posed significant challenges, requiring substantial GPU memory and extended training time. On CPU, batch sizes and throughput were severely constrained.

Additionally, due to the lack of off-the-shelf implementations for music-aware attention or relative positional embeddings, we retained absolute RoPE, potentially limiting the model's ability to capture long-range musical dependencies. Our current Mixture-of-Experts (MoE) module is applied externally, post-Transformer. While integrating MoE within earlier layers may allow for richer feature learning, the architectural changes required were non-trivial. The modularity constraints of the original codebase made such modifications impractical, and retraining from scratch under these adjustments was not feasible within our computational budget and project timeline.

Conclusion

In conclusion, the `text2midi` model is a very innovative work, as speech processing and generative models, in general, are part of an exponentially growing industry. Excluding some difficulties that we encountered while conducting the various experiments on this model, we have come to some important conclusions, which we believe will prove to be very helpful in developing the model and improving it to gain greater performance.

References

"Text2midi: Generating Symbolic Music from Captions": Keshav Bhandari, Abhinaba Roy, Kyra Wang, Geeta Puri, Simon Colton, Dorien Herremans.